

1. Fundamentos da Web e Protocolo HTTP

1.1 Estrutura de um URL (Uniform Resource Locator)

Um URL é constituído por:

- **Scheme** (ou protocolo): `http`, `https`
- **Host**: `www.example.com`
- **Porta** (opcional): `:8080`
- **Path**: `/products/123`
- **Query string** (opcional): `?id=123&category=electronics`
- **Fragmento** (opcional): `#section2`

Exemplo:

```
http://www.example.com:8080/products?id=123&category=electronics#details
```

- **Path** → `/products`
- **Query string** → `id=123&category=electronics` (parâmetros `id` e `category`)
- **Fragmento** → `details`

1.2 Métodos HTTP

Método	Safe	Idempotente	Tem body?	Uso típico
GET	✓ Sim	✓ Sim	Não	Obter recurso
POST	✗ Não	✗ Não	Sim	Criar recurso (URI não conhecido previamente)
PUT	✗ Não	✓ Sim	Sim	Criar/substituir recurso (URI conhecido previamente)
DELETE	✗ Não	✓ Sim	Opcional	Remover recurso

Definições formais:

- **Safe (seguro)** → não altera o estado da aplicação (operação *read-only*).
- **Idempotente** → o efeito de múltiplos pedidos idênticos é o mesmo que o de um único pedido.

1.3 Status Codes HTTP (séries)

Série	Significado	Exemplos
2xx	Sucesso	200 OK, 201 Created
3xx	Redirecionamento	303 See Other
4xx	Erro do cliente	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Erro do servidor	500 Internal Server Error, 502 Bad Gateway

Status codes específicos a conhecer:

- **200 OK** → pedido processado com sucesso.
- **201 Created** → recurso criado com sucesso.
- **303 See Other** → browser deve fazer um pedido GET para o URL no header **Location**.
- **400 Bad Request** → pedido mal formado; parâmetro obrigatório em falta (query string ou body).
- **401 Unauthorized** → credenciais inválidas ou ausentes.
- **403 Forbidden** → credenciais válidas mas sem permissão para aceder ao recurso.
- **404 Not Found** → recurso não existe.
- **500 Internal Server Error** → erro do lado do servidor (ex: base de dados indisponível).

1.4 Headers HTTP importantes

Header	Direção	Propósito
Content-Type	Pedido ou Resposta	Tipo de conteúdo do corpo (application/json , text/html , application/csv)
Authorization	Pedido	Credenciais de autenticação (ex: token JWT)
Location	Resposta (3xx)	URL para onde redirecionar
Cookie	Pedido	Enviar cookies armazenados pelo cliente
Set-Cookie	Resposta	Servidor guarda dados no cliente

1.5 Parâmetros num pedido HTTP

Um pedido pode ser parametrizado através de:

1. **Query string** → **?name=value**
2. **Request body** → corpo do pedido (JSON, formulário, CSV, etc.)
3. **Request headers** → headers HTTP
4. **Path (URI)** → **/users/:id**

Não é uma forma válida: **middleware arguments** (isto é um conceito do Express, não do HTTP).

2. HTML e CSS (lado do cliente)

2.1 Estrutura de um documento HTML

- Documento é constituído por **elementos** e **texto**.
- Elementos têm **marca de início** e, opcionalmente, **marca de fim**.
- Na marca de início podem existir **atributos**.
- Atributos têm **nome** e podem ter **valor**.
- Elemento raiz é **<html>**.
- **<head>** contém metadados (título, links para CSS/JS).
- **<body>** contém conteúdo visível.

2.2 Formulários HTML (elemento **<form>**)

- Submissão automática com botão de submit:

```
<form method="post">
  <input type="submit" value="Enviar">
</form>
```

- Atributo `method` do formulário define o verbo HTTP (`post` ou `get`).
- Botão de submissão define-se com `<input type="submit">` ou `<button type="submit">`.

2.3 Seletores CSS (em JavaScript)

Seletor	Exemplo	O que seleciona
Por ID	<code>#user1</code>	Elemento com <code>id="user1"</code>
Por classe	<code>.note</code>	Todos os elementos com <code>class="note"</code>
Por tipo	<code>div</code>	Todos os elementos <code><div></code>
Classe + tipo	<code>div.special</code>	<code><div></code> com <code>class="special"</code>

Métodos JavaScript para obter referências a elementos:

- `document.querySelector(selector)` → retorna o primeiro elemento correspondente.
- `document.querySelectorAll(selector)` → retorna todos (NodeList).

Exemplos corretos:

- `document.querySelectorAll(".note")` → todos os elementos com classe `note`.
- `document.querySelectorAll("button")` → todos os elementos `<button>`.
- `document.querySelector("#user1")` → elemento com id `user1`.

3. JavaScript Fundamentos

3.1 Funções em JavaScript

- Funções de **ordem superior** → recebem ou retornam outras funções.
- **Closure** → capacidade de uma função "capturar" o ambiente léxico onde foi definida, permitindo aceder a variáveis dessa envolvente mesmo depois da função exterior ter retornado.
- **Callback** → função passada como argumento a outra função.
- **Arrow function** → sintaxe concisa; não tem o seu próprio `this`.

3.2 A palavra-chave `this`

O valor de `this` depende de como a função é chamada:

Contexto de chamada	Valor de <code>this</code>
Como função global	<code>undefined</code> (strict mode) ou objeto global
Como método de um objeto	O objeto usado na chamada

Contexto de chamada	Valor de <code>this</code>
Como função construtora (<code>new</code>)	O novo objeto criado
Através de <code>call()</code> ou <code>apply()</code>	O objeto arbitrário passado como argumento

3.3 Funções `async`

Uma função **tem obrigatoriamente a palavra `async`** na assinatura se:

- No corpo da função for usada a palavra `await`.

[VERIFICAR: Uma função que retorna uma *Promise* não precisa obrigatoriamente de `async` — pode retornar explicitamente `new Promise()`]

3.4 Métodos de Array

Método	Retorna	Descrição
<code>map()</code>	Novo array com mesmo número de elementos	Transforma cada elemento
<code>filter()</code>	Novo array com menos ou igual elementos	Remove elementos com base em predicado
<code>find()</code>	Um elemento ou <code>undefined</code>	Primeiro elemento que satisfaz predicado
<code>forEach()</code>	<code>undefined</code>	Itera sobre cada elemento
<code>pop()</code>	O elemento removido (do final)	Remove último elemento

3.5 Objetos e propriedades

- `{prop: [1].pop}` → cria um objeto com uma propriedade `prop` cujo valor é **a função `pop`** (não o resultado da sua execução).

4. Assincronia em JavaScript

4.1 Modelo de execução (Event Loop)

JavaScript é **single-threaded** com loop de eventos. Funções assíncronas (`setTimeout`, `fetch`) são colocadas na fila de tarefas.

- Um `while (condição) { /* do nothing */ }` bloqueia a thread principal, impedindo a execução de qualquer callback pendente, mesmo que o `setTimeout` já tenha expirado.

Consequência prática: Se houver um ciclo infinito ou muito longo, callbacks agendados nunca são executados.

4.2 Promises

Uma Promise representa um valor que estará disponível no futuro (ou uma rejeição).

Estados:

- `pending` → pendente
- `fulfilled` → resolvida com sucesso
- `rejected` → rejeitada com erro

Métodos principais:

Método	Propósito
<code>then(onFulfilled, onRejected)</code>	Regista callback para sucesso (retorna nova Promise)
<code>catch(onRejected)</code>	Regista callback para rejeição
<code>finally(onFinally)</code>	Executa callback independentemente do resultado
<code>Promise.all(iterable)</code>	Retorna Promise que resolve quando todas resolvem; rejeita se alguma rejeitar
<code>Promise.allSettled(iterable)</code>	Retorna Promise que resolve quando todas terminam (com sucesso ou rejeição); nunca rejeita
<code>Promise.any(iterable)</code>	Retorna Promise que resolve com o valor da primeira que resolve; rejeita se todas rejeitarem

`Promise.allSettled` — formato do resultado:

```
[
  { status: "fulfilled", value: 33 },
  { status: "rejected", value: Error("an error") }
]
```

4.3 `async/await`

- `await` só pode ser usado dentro de uma função marcada com `async`.
- `await` pausa a execução da função até a Promise ser resolvida ou rejeitada.
- Uma função `async` retorna sempre uma Promise (o valor de retorno é automaticamente envolvido numa Promise).

Relação com Promises: Qualquer código com `async/await` pode ser reescrito usando explicitamente `.then()` e `.catch()`.

4.4 Exemplo: transformação de `async/await` para Promises

```
// Com async/await
async function f3() {
  const num = await f1();
  return await f2() + ":" + num;
}
```

```
// Com Promises
function f3() {
  return f1().then(num => f2().then(res => res + ":" + num));
}
```

5. Node.js

5.1 O que é Node.js?

Node.js é um **ambiente de execução JavaScript** do lado do servidor, baseado no motor V8 do Chrome.

5.2 Ficheiro `package.json`

Papéis do `package.json`:

- Registrar as dependências da aplicação.
- Distinguir dependências de execução (`dependencies`) de dependências de desenvolvimento (`devDependencies`).
- Definir comandos de execução (scripts): `npm start`, `npm test`, etc.

Não é papel do `package.json`: **Armazenar objetos JSON recebidos nos pedidos HTTP.**

5.3 Módulos `fs` (file system)

- `fs.readFile()` → leitura assíncrona de ficheiros.
- Operações de I/O em Node.js devem ser assíncronas para não bloquear o event loop.
- [VERIFICAR: Funções síncronas existem (`fs.readFileSync`), mas bloqueiam a thread.]

6. Express.js — Framework Web para Node.js

6.1 Conceitos fundamentais

- **Aplicação Express** → instância criada com `express()`.
- **Rota** → identificada por **path do URI + método HTTP**.
- **Handler** → função que processa um pedido e produz resposta.
- **Middleware** → função com acesso aos objetos `req`, `res` e `next`. Pode:
 - Executar código.
 - Modificar `req` e `res`.
 - Terminar o ciclo (enviar resposta).
 - Chamar o próximo middleware com `next()`.
 - **Interceptar um pedido antes de chegar ao handler final.**

6.2 Registrar rotas

```
app.get("/users/:id", handler1);
app.post("/items", handler2);
```

```
app.put("/items/:id", handler3);
app.delete("/users/:username", handler4);
```

Parâmetros de path → `:id`, `:username` — acedidos em `req.params`.

6.3 Objeto `Request` (req)

Propriedade/método	Conteúdo
<code>req.params</code>	Parâmetros da path (<code>/users/:id</code> → <code>req.params.id</code>)
<code>req.query</code>	Parâmetros da query string (<code>?page=5</code> → <code>req.query.page</code>)
<code>req.body</code>	Corpo do pedido (requer middleware de parsing)
<code>req.get(headerName)</code>	Valor de um header HTTP (<code>req.get("Content-Type")</code>)
<code>req.path</code>	Path do URI (<code>/users/123</code>)

6.4 Middleware

Registrar middleware global:

```
app.use(middlewareFunction);
```

Registrar middleware para um path específico:

```
app.use("/api", middlewareFunction);
```

Middleware para parsing de JSON:

```
app.use(express.json());
```

- Regista o **retorno da função** `express.json()` como middleware.
- Coloca na propriedade `req.body` o conteúdo do corpo do pedido com `Content-Type: application/json`.

Middleware para parsing de formulários URL-encoded:

```
app.use(express.urlencoded({ extended: true }));
```

6.5 Exemplo de construção de um middleware

Uma função que **retorna um middleware** (factory pattern):

```
function loggerConstructor(headers = []) {  
  return function (req, res, next) {  
    let str = `${req.method} - ${req.path}`;  
    if (headers.length) {  
      str += "\n" + headers.map(h => `${h}: ${req.get(h) || ""}`).join("\n");  
    }  
    console.log(str);  
    next();  
  };  
}
```

6.6 Tratamento de erros em middleware

Para evitar que o servidor termine com erro (ex: ficheiro inexistente), deve-se capturar rejeições de Promises e responder adequadamente:

```
fs.readFile(req.path)  
  .then(data => res.send(data))  
  .catch(err => res.status(500).send("Erro ao ler ficheiro"));
```

6.7 Status codes em Express

- `res.status(400).json({ error: "Bad request" })`
- `res.status(404).send("Not found")`
- `res.status(500).send("Internal error")`
- `res.status(201).json(newResource)`
- `res.status(303).location("/some/url").send()`

7. Cliente HTTP em JavaScript

7.1 Fetch API

- `fetch(url, options)` → retorna uma Promise que resolve com um objeto `Response`.
- `options` pode incluir: `method`, `headers`, `body`.

Exemplo:

```
fetch("https://api.example.com/data", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ name: "John" })  
})  
  .then(response => response.json())  
  .then(data => console.log(data));
```


7.2 Objeto `Response`

- `response.json()` → retorna Promise que resolve com o corpo interpretado como JSON.
- `response.text()` → retorna Promise com o corpo como texto.
- `response.status` → status code da resposta.

7.3 Headers em pedidos fetch

- `Content-Type` → tipo de conteúdo enviado.
- `Authorization` → credenciais (ex: `Bearer token`).

8. Handlebars (Motor de Templates)

8.1 Conceitos básicos

- **Entradas** → um template + um objeto (contexto).
- **Saída** → HTML gerado.
- **Template** → contém **texto** e **expressões Handlebars** (`{{ }}`).
- **Expressões** acedem a **propriedades do objeto** de contexto.

8.2 Expressões Handlebars importantes

Expressão	Propósito
<code>{{#each array}}</code>	Itera sobre uma lista
<code>{{#with obj}}</code>	Altera o contexto do bloco
<code>{{#unless cond}}</code>	Renderiza bloco condicionalmente (se cond for falsy)
<code>{{@index}}</code>	Índice da iteração atual (dentro de <code>#each</code>)

9. Autenticação e Sessões

9.1 Header `Authorization`

No contexto dos trabalhos práticos, o header `Authorization` foi usado para enviar:

- O **token associado ao utilizador** (ex: JWT), após autenticação inicial.

9.2 Passport.js

- Middleware para autenticação em Node.js/Express.
- Suporta várias estratégias (local, JWT, OAuth, etc.).

9.3 Cookies

- Servem para **o servidor guardar dados no cliente e obtê-los futuramente**.
- Enviados do servidor para o cliente via header `Set-Cookie`.
- Enviados do cliente para o servidor via header `Cookie`.

10. Arquitetura de aplicações web (práticas laboratoriais)

10.1 Estrutura de módulos sugerida

Módulo	Responsabilidade
*-api	Recebe objetos <code>Request</code> e <code>Response</code> do Express; orquestra pedidos
*-services	Contém a lógica de negócio; valida dados; chama módulos de dados
*-data	Acesso a dados (base de dados, APIs externas)

Princípios:

- O módulo `*-services` **nunca deve receber objetos `Request` ou `Response`** do Express.
- O módulo `*-api` deve extrair os dados do pedido e chamar `*-services`.

10.2 Operações em paralelo com Promises

Para realizar múltiplas operações assíncronas em paralelo e só depois prosseguir:

- `Promise.all()` → quando todas devem ter sucesso.
- `Promise.allSettled()` → quando se quer o resultado de todas, mesmo com falhas.
- `Promise.any()` → quando se quer a primeira bem-sucedida.

Exemplo de remoção em paralelo:

```
function removeUserGroups(userId, groupsIds) {  
  const promises = groupsIds.map(groupId => removeGroup(userId, groupId));  
  return Promise.all(promises);  
}
```

11. Tópicos Avançados e Boas Práticas

11.1 Substituir implementação de `fetch` (monkey patching)

Para contar pedidos ou adicionar logging, pode-se substituir a função global `fetch`:

```
const originalFetch = fetch;  
fetch = function(uri, options) {  
  const method = options?.method || "GET";  
  COUNTERS[method]++;  
  return originalFetch(uri, options);  
};
```

11.2 Middleware para parsing de CSV

- `Content-Type: application/csv`.

- Converter corpo do pedido (texto) em array de arrays (linhas x colunas).
- Opcional: primeira linha como cabeçalho (`columns`).

11.3 `this` em funções que usam `Array.prototype.pop`

Quando se atribui `items.pop` a uma propriedade de um objeto, perde-se o contexto (`this`). Para resolver:

- Usar `bind`: `this.removeItem = items.pop.bind(items)`
- Ou usar arrow function: `this.removeItem = () => items.pop()`

Termos-chave (para revisão)

- HTTP, URL, URI, scheme, host, path, query string, fragmento
- Métodos HTTP: GET, POST, PUT, DELETE
- Safe, idempotente
- Status codes: 200, 201, 303, 400, 401, 403, 404, 500, 502
- Headers: Content-Type, Authorization, Location, Cookie, Set-Cookie
- HTML, elemento, atributo, formulário, input submit
- CSS, seletor de classe (`.`), seletor de ID (`#`), seletor de tipo
- JavaScript: closure, callback, this, arrow function, função async, await
- Promise, then, catch, finally, Promise.all, Promise.allSettled, Promise.any
- Node.js, package.json, fs, módulos
- Express: app, rota, handler, middleware, req, res, next, req.params, req.query, req.body, app.use, express.json, express.urlencoded
- Fetch API, Response, body JSON
- Handlebars: template, contexto, `#each`, `#with`, `#unless`, `@index`
- Autenticação: Authorization, token, Passport.js
- Cookie, Set-Cookie
- Arquitetura: api, services, data
- Event loop, bloqueio, setTimeout
- Monkey patching, bind

[ORDEM INCERTA] Os seguintes tópicos aparecem em testes mas com menor frequência ou de forma menos sistemática, podendo a ordem relativa ser ajustada:

- Passport.js (autenticação) — surge em contexto de middleware mas com detalhes limitados nos testes.
- Tratamento de erros com `fs.readFile` e respostas HTTP adequadas.
- Construção de middlewares que devolvem funções (factory pattern).